

i DAT516/DAT690/DIT516 Exam Instructions

Write your answers in Inspera. You can of course use separate sheets of paper to draft and experiment, but only the answers in Inspera will be graded. The answers can and should be short.

Python interpreters such as Self Practice are not available. Hence you have to reason about the answers on your own. Using pencil and paper is a good aid for this.

You can also have with you an A4 paper, hand-written on both sides, with whatever information you want to have available

You will need 24 points out of 60 in this exam to get accepted with grade 3, 36 points for grade 4, and 48 points for grade 5. The exam grade will also be the final grade for the course.

If used as a re-exam for DAT515 or DIT515 from earlier years, the final grade of the course will be the grade from your labs, provided that you get at least 24 points in this exam.

i Python Syntax

For your reference: the syntax of (the relevant parts of) Python is attached.

1 Question 1: types and values

Question 1 (16 p). What is the value and type of the following expressions? Remember that **None** is also a value! It can also happen that the expression contains an error. In that case, indicate the kind of error: one of `SyntaxError`, `TypeError`, `AttributeError`, `ZeroDivisionError`, `KeyError`, `NameError`, `IndexError`, and explain (in your own words) the reason of the error. (1+1 p for each part)

`{4,5,6}.add(5)`

Fill in your answer here

`{n%3 for n in range(1, 10, 3)}`

Fill in your answer here

`set([[1, 2, 2, 1]])`

Fill in your answer here

`(lambda x, y: y(x, x+1))(2, max)`

Fill in your answer here

`set([1, 2, 2, 1])`

Fill in your answer here

`'a cat' is 'a cat'`

Fill in your answer here

`{m: m%3 in range(1,4)}`

Fill in your answer here

`len({print(n) for n in range(1, 100) if 10 < n < 20})`

Fill in your answer here

Maximum marks: 16

2 Question 2: comprehensions

```
nobel_data = [
    {"person": "Robert Robinson",
     "sex": "male",
     "award": "Nobel Prize in Chemistry",
     "year": 1947,
     "birthDate": 1886,
     "birthPlace": "Rufford",
     "country": "United Kingdom",
     "deathDate": 1975
    },
    {"person": "Han Kang",
     "sex": "female",
     "award": "Nobel Prize in Literature",
     "year": 2024,
     "birthDate": 1970,
     "birthPlace": "Gwangju",
     "country": "South Korea"
    }, # etc
]
```

Question 2 (12 p). The code above shows a sample from the list `nobel_data`, of Nobel prize laureates obtained from Wikidata. All items in the list have the same labels, except for `deathDate`, which is not given for persons that are still alive.

Your task is now to find answers to some questions by analysing `nobel_data`. We assume that you have access to the whole list, which in fact contains 935 items. Every answer must be a valid Python expression, such as a comprehension or a function application - not a statement such as a for loop.

Question 2.1, 3 points: write an expression that returns the number of different awards (Chemistry, Literature, etc).

Write your answer here.

Question 2.2, 3 points: the names of all female laureates.

Write your answer here.

Question 2.3, 3 points: the number of persons who are still alive.

Write your answer here.

Question 2.4, 3 points: the number of laureates per country, given as a dictionary where countries are keys.

Write your answer here.

Maximum marks: 12

3 Question 3: classes and inheritance

Question 3 (16p)

The following class `Year` represents years for historical purposes, distinguishing between Before the Common Era (BCE, i.e. before year 0) and after. Notice, in particular, the dunder method `__lt__`, which defines the ordering relation of years.

```
class Year:
    def __init__(self, year, is_bce=False):
        self.year = year
        self.is_bce = is_bce
    def normalise(self):
        return self.year * (-1 if self.is_bce else 1)
    def __str__(self):
        return str(self.year) + (' BCE' if self.is_bce else '')
    def __lt__(self, other):
        return self.normalise() < other.normalise()
```

3.1 (4p)

Given the two years below:

```
y1 = Year(1954)          # birth of Sócrates, Brazilian footballer
y2 = Year(470, True)     # birth of Socrates, Greek philosopher
```

What is printed by the following?

```
print(y1, '<', y2, y1 < y2)
```

3.2 (6p)

Define a class `YearSpan` for representing a span of time between two years. It should take two `Year` objects as arguments to its initialisation method. Its `__str__` method should print the years in order (earliest first) together with the number of years in between, i.e.:

```
>>> print(YearSpan(y1, y2))
470 BCE -> 1954 = 2424 years
```

3.3 (6p)

Now define another class called `EventYear` for representing a year together with a description of a particular event that happened in it. The event description should be the third argument to the initialisation method, and the `__str__` method should also include this description, as shown below. This class should inherit from `Year` and reuse as much as possible from it. In particular, you should be able to use `EventYear` with `YearSpan` without any modifications necessary, i.e.:

```
>>> ey1 = EventYear(200, True, 'Invention of paper')
>>> ey2 = EventYear(1989, False, 'Invention of internet')
>>> print(ey1)
200 BCE: Invention of paper
```

```
>>> print(ey2)
1989: Invention of internet
>>> print(YearSpan(ey1, ey2))
200 BCE: Invention of paper -> 1989: Invention of internet = 2189 years
```

Maximum marks: 16

4 Question 4: library API

Question 4 (16p)

Attached is the API for a (fictitious) Python library for making HTTP requests. All answers in this section should make use of this API, respecting the documented types. Your code does not need to include type hints.

4.1 (4p)

Send a GET request to `https://example.com` with query string `user=42`. Store the response in a variable named `resp` if its status code is 200. Otherwise, `resp` should be set to `None`.

Fill in your answer here

4.2 (4p)

Check if your response from 4.1 contains a header with key `SessionID`. If so, store the corresponding header value in a variable named `sid`.

Fill in your answer here

4.3 (4p)

Now send a POST request to `https://example.com/logout`. This request should have a header with key `SessionID` and value obtained from 4.2 above.

Fill in your answer here

4.4 (4p)

Write an endless loop which enumerates over user IDs from 1 upwards and for each user ID repeats the steps above, i.e.:

1. Make a GET request with the user ID in the query string (4.1)
2. Get the session ID from the corresponding response (4.2)
3. Send a POST request to logout the user with that session ID (4.3)

Make sure to handle the errors mentioned above (non-200 status codes and missing session headers). The loop should continue with the next user ID when such an error is encountered.

Fill in your answer here

Maximum marks: 16

Document 2

Attached



```

<stm>      ::= <decorator>* class <name> (<name>,*):? <block>
            | <decorator>* def <name> (<arg>,*): <block>
            | import <name> <asname>?
            | from <name> import <imports>
            | <exp>,* = <exp>,*
            | <exp> <assignop> <exp>
            | for <name> in <exp>: <block>
            | <exp>
            | return <exp>,*
            | yield <exp>,*
            | if <exp>: <block> <elses>?
            | while <exp>: <block>
            | pass
            | break
            | continue
            | try: <block> <except>* <elses> <finally>?
            | assert <exp> ,<exp>?
            | raise <name>
            | with <exp> as <name>: <block>

<decorator> ::= @ <exp>
<asname>    ::= as <name>
<imports>   ::= * | <name>,*
<elses>     ::= <elif>* else: <block>
<elif>      ::= elif <exp>: <block>
<except>    ::= except <name>: <block>
<finally>   ::= finally: <block>
<block>     ::= <stm> <stm>*
<exp>       ::= <exp> <op> <exp>
            | <name>?.?<name>(<arg>,* )
            | <literal>
            | <name>
            | ( <exp>,* )
            | [ <exp>,* ]
            | { <exp>,* }
            | <exp>[<exp>]
            | <exp>[<slice>,*]
            | lambda <name>*: <exp>
            | { <keyvalue>,* }
            | ( <exp> for <name> in <exp> <cond>? )
            | [ <exp> for <name> in <exp> <cond>? ]
            | { <exp> for <name> in <exp> <cond>? }
            | { <exp>: <exp> for <name> in <exp> <cond>? }
            | - <exp>
            | ~ <exp>
            | not <exp>
            | <name> := <exp>

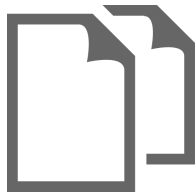
<keyvalue>  ::= <exp>: <exp>
<arg>       ::= <name>
            | <name> = <exp>
            | *<name>
            | **<name>

<cond>      ::= if <exp>
<op>        ::= + | - | * | ** | / | // | % | @ | == | > | >= | < | <= | !=
            | in | not in | and | or | & | | | ^ | << | >> | :=
<assignop>  ::= += | -= | *= | /= | %= | &= | |= | ^=
<slice>     ::= <exp>? :<exp>? <step>?
<step>      ::= :<exp>?

```

Question 4

Attached



Exam 2026-08 Q4 API

Type annotation syntax

Expression	Explanation
<code>x: T</code>	Variable <code>x</code> has type <code>T</code>
<code>def f(x: T) -> U</code>	Function <code>f</code> takes a parameter <code>x</code> of type <code>T</code> and returns a value of type <code>U</code>
<code>T U</code>	Either type <code>T</code> or type <code>U</code>
<code>list[T]</code>	A list of items of type <code>T</code>
<code>dict[T, U]</code>	A dictionary with keys of type <code>T</code> and values of type <code>U</code>
<code>tuple[T, U]</code>	A tuple with first item of type <code>T</code> and second item of type <code>U</code>

HTTP Library API

```
# A URL object
class URL:
    # initialise URL object from string
    def __init__(self, url: str): ...

    # e.g. 'http' or 'https'
    scheme: str

    # full domain name and path
    path: str

    # query string parameters as key/values
    params: dict[str, str]

# An HTTP request
class Request:
    # URL to send request to
    url: URL

    # HTTP method, e.g. GET/POST/PATCH
    method: str

    # request headers as a list of tuples of strings
    headers: list[tuple[str, str]]

    # request payload as raw string (or None)
    body: str | None

    # send this Request and return a Response
    def send(self) -> Response

# An HTTP response
class Response:
    # status code and text explanation, e.g. (404, "Not Found")
    status: tuple[int, str]

    # response headers as a list of tuples of strings
    headers: list[tuple[str, str]]

    # response payload as a raw string (or None)
    body: str | None
```

```

# Construct and return a Request object without sending it
def make_request(
    url: str | URL,
    method: str,
    headers: dict[str, str] | None = None,
    body: str | None = None
) -> Request

# Send an HTTP request, returning True if a response was received
def send_request(
    url: str | URL,
    method: str,
    headers: dict[str, str] | None,
    body: str | None
) -> bool

# Add a header to a Request object
def add_header(
    req: Request,
    key: str,
    value: str,
) -> None

# Get header corresponding to key in a Request or Response object
# Returns header as key/value pair, or None
def get_header(
    r: Request | Response,
    key: str
) -> tuple[str, str] | None

```